

Universidad Autónoma de Madrid

Escuela Politécnica Superior



Grado en Ingeniería Informática

TRABAJO DE FIN DE GRADO

TODO: TÍTULO DEL TRABAJO DE FIN DE GRADO

Javier López Cano
Tutor: Ramón Díaz Uriarte

Junio 2021

TODO: TÍTULO DEL TRABAJO DE FIN DE GRADO

Autor: Javier López Cano
Tutor: Ramón Díaz Uriarte

Dpto. de Bioquímica
Escuela Politécnica Superior
Universidad Autónoma de Madrid

Junio 2021

Agradecimientos

TODO: Agradecimientos.

“TODO: Cita relevante” TODO: Autor de la cita

Abstract

Abstract — TODO: Resumen en inglés, 250-500 palabras.

Key words — TODO: Palabras clave en inglés, separadas por coma.

Resumen

Resumen — TODO: Resumen en español, 250-500 palabras.

Palabras clave — TODO: Palabras clave en español, separadas por coma.

Índice general

1. Introducción	1
1.1. Alcance	1
1.2. Estructura del documento	1
2. Estado del Arte	3
3. Diseño	5
3.1. Definición de variables y reglas	5
3.2. Algoritmo de actualización	7
3.3. Definición de la estructura de salida	7
4. Desarrollo	9
4.1. Unificación de Ramas	9
4.2. Corrección de errores previos y optimización del código	10
4.3. Implementación en R	11
4.4. Implementación en C++	11
4.5. Integración en el flujo principal de código	11
4.6. Inclusión de las variables en las intervenciones	12
5. Resultados	13
6. Conclusiones y trabajo futuro	15
6.1. Conclusiones	15
6.2. Líneas de trabajo futuro	15
Apéndices	17
A. Ejemplos de bloques y comandos útiles en LaTeX	19
A.1. Ejemplo de sección	19

Índice de tablas

Índice de figuras

A.1. Logo de la Universidad Autónoma de madrid.	20
---	----

1

Introducción

1.1. Motivación

- ¿Intro sobre el cáncer e intervenciones?
- ¿Terapia adaptativa?

1.2. Objetivos

- Familiarizarse con OncoSimulR (especialmente el flujo general de simulación y las intervenciones)
- Aprender sobre R y C++
- Completar el merge de las ramas anteriores solucionando todos los errores que surjan
- ¿Optimizar el código de intervenciones?
- Añadir la posibilidad de definir variables de usuario arbitrariamente complejas tanto como salida en sí mismas como para emplearse en las intervenciones
- Generar una batería de pruebas para comprobar el correcto funcionamiento de las variables de usuario y las intervenciones que las emplean

1.3. Estructura del documento

TODO: Descripción de la estructura del documento

2

Estado del Arte

3

Diseño

3.1. Definición de variables y reglas

A la hora de plantearse el diseño necesario para esta funcionalidad, lo primero que necesitamos saber es qué son y como se definen las variables de usuario que vamos a implementar.

Las variables de usuario son variables arbitrariamente complejas que un usuario puede definir a partir de combinaciones o funciones de algunas variables ya existentes en el programa. Para obtener estas variables arbitrariamente complejas vamos a necesitar de una serie de reglas que determinen el valor de estas variables en cada momento de acuerdo a las condiciones propuestas por el usuario para cada una de las variables.

Por tanto podemos determinar que por un lado tendremos un listado de variables, cada una de las cuales deberá contener:

- Nombre: nombre de la variable de usuario que permite identificarla y diferenciarla de el resto de variables definidas.
- Valor: valor numérico de la variable en cada momento.

Por otro lado tendremos un listado con las reglas que permiten modificar y actualizar las distintas variables definidas. Estas reglas cuentan de:

- ID: identificador único de cada regla que permite diferenciarla del resto de reglas definidas.

- Condition: la condición que, si se cumple, hará que la variable de usuario se modifique.
- Action: acción que se llevará a cabo en caso de que la Condition se cumpla, es decir, es el conjunto de modificaciones a realizar a las variables de usuario en caso de que se cumpla la condición de la regla. podrá contener una o varias fórmulas que determinarán los nuevos valores de una o varias variables de usuario.

En este punto debemos mencionar también, qué variables del programa se ha decidido que sean empleables para la modificación de variables y definición de las reglas por parte del usuario y los motivos por los que se ha decidido esto. Las variables disponibles para la definición de las reglas son:

- birthRate: la tasa de nacimiento correspondiente a cada genotipo, es decir, es un número que determina la probabilidad que tiene una célula con este genotipo de reproducirse, y por tanto aumentar su número total en 1 en un ciclo de tiempo.
- deathRate: la tasa de muerte correspondiente a cada genotipo, es decir, es un número que determina la probabilidad que tiene una célula con este genotipo de morir, y por tanto reducir su número total en 1 en un ciclo de tiempo.
- mutationRate: la tasa de mutación correspondiente a cada genotipo, es decir, es un número que determina la probabilidad que tiene una célula con este genotipo de mutar, y por tanto variar alguno de sus genes, pasando de ser de este genotipo a ser de otro genotipo distinto en 1 en un ciclo de tiempo.
- N: número total de células en la simulación, teniendo en cuenta todos los genotipos existentes.
- n_x: número de células de un determinado genotipo concreto, por ejemplo, n_A será el número de células del genotipo A, y n_A_B las del genotipo A_B
- T: tiempo transcurrido desde el inicio de la simulación, en segundos, con precisión de milisegundos.
- Variables de usuario: otras variables de usuario definidas en la misma simulación.

Estas variables se han añadido con la finalidad de dar a las variables de usuario la mayor flexibilidad posible para que el usuario las defina de acuerdo a sus necesidades, considerando todas las variables interesantes tanto para la terapia adaptativa como desde un punto de vista biológico para estudiar su evolución.

Las variables N, n_x y T se consideran necesarias para la terapia adaptativa, ya que permiten definir variables de usuario que se modifiquen de acuerdo a la evolución del tamaño del tumor (N), a la cantidad de las células de un determinado tipo, que se consideren más perjudiciales o sensibles a un determinado tratamiento (n_x) o al tiempo transcurrido (T). Estas variables definidas de acuerdo a estos atributos se podrían emplear

luego para definir las intervenciones que permite el programa OncoSimulR, permitiendo adaptarlas de forma adecuada a las distintas combinaciones posibles de estos 3 atributos a lo largo de la simulación, generando así la posibilidad de simular terapia adaptativa. A estas 3 variables se les suman las propias variables de usuario, para poder así generar combinaciones aún más complejas que permitan simular escenarios altamente enrevesados.

Las otras variables birthRate, deathRate y mutationRate, se han añadido con una finalidad más informativa, ya que, a pesar de no ser tan interesantes desde el punto de vista de la terapia adaptativa (aunque pueden aplicarse en algunos casos), dan una información importante acerca de la adaptabilidad de los distintos genotipos a lo largo de la simulación, haciendo que el estudio de la evolución de variables de usuario que sean combinación de estas variables a lo largo de la simulación pueda tener un gran interés para el estudio tumoral.

3.2. Algoritmo de actualización

El algoritmo de actualización de las variables de usuario es a priori, bastante sencillo, pero su principal complejidad radica en su implementación e integración en el flujo de código principal, como veremos en la sección 4.

En rasgos generales el algoritmo de actualización es:

- El usuario define las variables de usuario que desea y las reglas para modificarlas
- Se verifica que están definidas correctamente
- Cada sampleEvery:
 - para cada regla:
 - Si se cumple la Condition:
 - ◇ Modificamos el valor de las variables de acuerdo al Action
 - Se pasa a la siguiente regla

3.3. Definición de la estructura de salida

- Se modifica la estructura de salida de la función OncoSimulIndiv en R añadiendo los campos:
 - x\$other\$userVarNames lista con los nombres (ids") de las variables de usuario creadas
 - x\$other\$userVarValues matriz con los valores de las variables de usuario en cada sampleEvery (en el mismo orden que los nombres de las variables en x\$other\$userVarNames)

- Finalidad: poder emplear las variables de usuario (que son arbitrariamente complejas) como datos de salida y no solo como valores a emplear en otros procesos como las intervenciones. Por ejemplo graficar la evolución en el tiempo de algunas variables de usuario creadas puede ser interesante para evaluar la evolución del tumor en una simulación.

4

Desarrollo

4.1. Unificación de Ramas

- Intro ramas alberto y javier ¿Aquí o en estado del arte?
- Merge entre las ramas de Alberto y Javier (se hizo a mano por el gran número de problemas que producía el merge automático, los cambios se hicieron sobre la rama de Alberto por ser la que más cambios introducía sobre el código "base" del programa de simulación):
 - Modificación de código de Javier en la función UpdatePopulations de interventions.cpp para que emplee birth y death rate como la rama de Alberto
 - Modificación de los parametros de entrada de todas las funciones de OncoSimulR para que tomen de forma adecuada el parámetro de intervenciones, así como actualización de RcppExports.R, RcppExports.cpp y OncoSimulR_init.c con el mismo fin. - Adición de los tests de intervenciones y los mans de intervenciones a la rama de alberto, así como interventions.cpp (con updatepopulations modificado), interventions.h, multivariant_hypergeometric.cpp y multivariant_hypergeometric.h
- Merge automático con los últimos cambios muy simples que introdujo Ramón sobre el programa en ese tiempo.

4.2. Corrección de errores previos y optimización del código

- Tras realizar el merge entre las ramas anteriores se detectaron una serie de fallos de la funcionalidad previa en los tests que fue necesario solucionar para proceder a la implementación de la nueva funcionalidad. Estos fallos estaban localizados en los tests de la funcionalidad de intervenciones que se ha aplicado anteriormente (o bien en estado del arte o en la sección de unificación de ramas).
- Para solucionar los fallos:
 - Añadir `InterventionTimes` a `x$other$interventionTimes` con los tiempos de las intervenciones para poder realizar tests (se rellena en `BNB_nr.cpp` tras haber cambiado `executeInterventions`)
 - Adelantar la ejecución de intervenciones para que se realice antes de rellenar las estructuras de salida para evitar que las actualizaciones de población de las intervenciones se realicen 1 `samboleEvery` después de lo debido y para que la tabla de poblaciones y la de birth y death rates estén correctamente actualizadas y sincronizadas entre sí
 - Comentar la comprobación de `totPopSize != totalPop` en `reduceTotalPopulation` (`interventions.cpp`) porque al haberlo adelantado `totPopSize` no está actualizado
 - Añadir código para eliminar los genotipos con 0 población después de realizar intervenciones para evitar el error que se obtenía: `Unrecoverable exception: ti: popSize <= 0. spP.popSize = 0.000000. Aborting.`
 - Adaptar los tests de intervenciones a estos nuevos cambios y comprobar que todos los tests se pasan correctamente
- Tras esto además realizamos algunos cambios en el código pues durante estas modificaciones se vio que en numerosas ocasiones se ejecutaban fragmentos de código innecesarios, que en una ejecución de una simulación de gran tamaño suponían un gran aumento del tiempo de ejecución, para ello:
 - Modificamos la función `ExecuteInterventions` de `interventions.cpp`:
 - Hacer que devuelva `true` si hay intervención y `false` si no y en caso de error solo lance excepción (esto permite el siguiente punto y permite obtener `InterventionTimes` desde fuera)
 - Hacer que solo se llame a `UpdatePopulations` si ha habido intervención (antes se ejecutaba en todas las iteraciones aunque no hubiese intervenciones)
 - Cambiar `ParseWhatHappens`, ahora devuelve `void` y si hay error solo lanza excepción

4.3. Implementación en R

- creación de variables y reglas en R
- verificación de:
 - no hay 2 uservars con mismo nombre
 - no hay 2 reglas con mismo id
 - están bien especificadas (reglas y variables tienen todos los atributos, las reglas tienen la estructura correcta en condition y action)
 - Modificación de los n_x en condition y action de las reglas
- Modificación de la llamada a oncoSimulIndiv añadiendo userVars y rules

4.4. Implementación en C++

- estructuras Rule y UserVarsInfo
- funciones de user_var.cpp:
 - Básicas (createRule, createUserVarsInfo, destroyRule, compareRules, addRule, printRule, printUserVarsInfo)
 - isValid (Comprueba si las expresiones de las acciones de las reglas están bien formadas)
 - executeRules (Comprueba si se cumple la condición mediante exprtk, y si se cumple llama a parseAction)
 - parseAction (compila la acción de la regla con exprtk y la ejecuta variando las variables de usuario necesarias en el map de UserVarsInfo)

4.5. Integración en el flujo principal de código

- Se incluye la ejecución de las reglas en el flujo de código de la simulación en la función nr_innerBNB de BNB_nr.cpp justo antes de la ejecución de las intervenciones para permitir que las variables de usuario estén actualizadas antes de realizar las intervenciones ya que se emplean para estas.
- Se crea la función evalFVarsFitness que permite acceder al birthrate, deathrate y mutationrate que se deben pasar como argumentos para executeRules para poder añadirse a la tabla de símbolos de exprtk y poder emplearse en las fórmulas para las codiciones o modificaciones de las reglas para actualizar las variables de usuario.

- Empleamos tambien la función `evalFVars` para acceder a las poblaciones de los genotipos con la misma finalidad.
- Modificamos la estructura de salida de la función `BNB_nr_Algo5` para que incluya la lista con los nombres de las variables de usuario y la matriz con los valores de estas en cada `sampleEvery`
- Modificamos las cabeceras y llamadas a funciones para que incluyan las nuevas variables `userVars`, `Rules` y `userVarsInfo`, y se realicen las llamadas de forma adecuada.
- Modificación de los parametros de entrada de todas las funciones de `OncoSimulR` para que tomen de forma adecuada el parámetro de intervenciones, así como actualización de `RcppExports.R`, `RcppExports.cpp` y `OncoSimulR_init.c` con el mismo fin.

4.6. Inclusión de las variables en las intervenciones

- Modificamos `executeInterventions` para que reciba como argumento la estructura `userVarsInfo` con los datos de las variables de usuario y añadimos un pequeño fragmento que añade las variables de usuario y sus valores a la tabla de símbolos de `exprtk` para poder ser empleadas en las intervenciones.
- Realizamos el mismo proceso para `ParseWhatHappens`.
- Modificamos las cabeceras y llamadas a estas 2 funciones de forma adecuada para que incluyan `userVarsInfo` en todas las llamadas.

5

Pruebas y Resultados

5.1. Comprobación de funcionalidad previa

- Comprobamos que la funcionalidad previa no se ha perdido ejecutando la batería de tests ya existentes para todo oncoSimulR con la herramienta testthat y tambien los manuales.

5.2. Tests de variables de usuario

- Creamos el fichero de tests para testthat test.user_var.R que contiene 8 tests para probar el correcto funcionamiento de las variables de usuario
 - Probar que las variables se crean correctamente
 - Comprobar que 2 variables no pueden tener el mismo nombre
 - Comprobar que las reglas se crean correctamente
 - Comprobar que 2 reglas no pueden tener el mismo ID
 - Comprobar que la ejecutarse las reglas las variables de usuario se modifican de forma correcta (en función de la variable T)
 - Comprobar que la ejecutarse las reglas las variables de usuario se modifican de forma correcta (en función de la variable N)
 - Comprobar que la ejecutarse las reglas las variables de usuario se modifican de forma correcta (en función de las poblaciones por genotipo n_x)

- Comprobar que la ejecutarse las reglas las variables de usuario se modifican de forma correcta (en función de la otras variables de usuario)

5.3. Tests de intervenciones

- Modificamos el fichero de tests de intervenciones `test-interventions.R` para incluir dos tests que compruebe que las intervenciones se ejecutan de forma adecuada cuando el trigger depende de variables de usuario y cuando la función de modificación emplea variables de usuario.

5.4. Resultados

- ¿Optmización?
- ¿?

6

Conclusiones y trabajo futuro

6.1. Conclusiones

6.2. Líneas de trabajo futuro

- Emplear variables de usuario en otras funcionalidades más allá de las intervenciones
- Permitir graficar variables de usuario en función del tiempo como una salida interesante de forma similar a la evolución de la población de los genotipos

Apéndices



Ejemplos de bloques y comandos útiles en LaTeX

A.1. Ejemplo de sección

La DARPA creó el protocolo de Internet [**ipv4sta**].

Citamos el acrónimo Field-Programmable Gate Array (FPGA).

Bitstream es una secuencia de bits.

La figura A.1 se utiliza en la portada.



Figura A.1: Logo de la Universidad Autónoma de madrid.

Código A.1: Algoritmo de ordenación Quicksort

```
#include <stdio.h>

void quick_sort (int *a, int n) {
    int i, j, p, t;
    if (n < 2)
        return;
    p = a[n / 2];
    for (i = 0, j = n - 1;; i++, j--) {
        while (a[i] < p)
            i++;
        while (p < a[j])
            j--;
        if (i >= j)
            break;
        t = a[i];
        a[i] = a[j];
        a[j] = t;
    }
    quick_sort(a, i);
    quick_sort(a + i, n - i);
}
```

```
#include <stdio.h>

void quick_sort (int *a, int n) {
    int i, j, p, t;
    if (n < 2)
        return;
    p = a[n / 2];
    for (i = 0, j = n - 1;; i++, j--) {
        while (a[i] < p)
            i++;
        while (p < a[j])
            j--;
        if (i >= j)
            break;
        t = a[i];
        a[i] = a[j];
        a[j] = t;
    }
    quick_sort(a, i);
    quick_sort(a + i, n - i);
}
```

La ecuación de Euler ($e^{\pm i\theta} = \cos \theta \pm i \sin \theta$) es citada frecuentemente como un ejemplo de belleza matemática.

$$a^2 + b^2 = c^2 \tag{A.1}$$